

Implementasi Deep Reinforcement Learning dengan Algoritma PPO pada Lingkungan Grid-Based untuk Permainan Nokia Snake Game

1. Pendahuluan

implementasi dan perbandingan dua algoritma Deep Reinforcement Learning Deep Q-Network (DQN) dan Proximal Policy Optimization (PPO) - untuk memainkan varian modern dari game Nokia Snake klasik pada grid 20×20. Kontribusi utama proyek adalah pemisahan antara faktor algoritma DQN vs PPO dan faktor arsitektur/state representation 12-bit vector vs 8-channel spatiotemporal tensor dengan Transformer Attention, sehingga perbandingan kausalitas menjadi apples to apples.

Varian target dinamis makanan yang bergerak dengan stochastic momentum dan greedy evasion ditambahkan untuk memaksa agen belajar perburuan target bergerak dan keluar dari jebakan Greedy Trap pola gagal klasik di mana agen pendekatannya terikat pada jalur lokal tanpa mempertimbangkan geometri global tubuh ular.

2. Tinjauan Pustaka

Judul Paper: [Deep Q-Snake: An Intelligent Agent Mastering the Snake Game with Deep Reinforcement Learning](#)

Penulis: Debjyoti Ray, Arindam Ghosh, Muneendra Ojha, Krishna Pratap Singh (IIIT Allahabad)

Tahun / Konferensi: 2024 / IEEE Region 10 Conference (TENCON)

Arsitektur dan State Space pada Paper Referensi:

Didefinisikan state space menggunakan himpunan 12-bit biner yang terdiri dari:

Status Makanan (4 bit): Makanan di Atas, Bawah, Kiri, Kanan.

Arah Agen (4 bit): Ular bergerak ke Atas, Bawah, Kiri, Kanan.

Status Dinding/Rintangan (4 bit): Terdapat dinding di Atas, Bawah, Kiri, Kanan.

Untuk memproses 12 fitur biner tersebut, Arsitektur neural network yang digunakan dalam paper dibangun menggunakan Keras dengan Model Sequential (Multi-Layer Perceptron / Dense Layers):

Input Layer: Menerima 12 digit state biner.

Hidden Layers: Terdiri dari dua lapis Dense/Fully Connected Layer menggunakan fungsi aktivasi ReLU untuk memperkenalkan non-linearitas.

Output Layer: Dense layer dengan 4 neuron yang merepresentasikan nilai Q-Value untuk masing-masing aksi diskrit (Atas, Bawah, Kiri, Kanan).

3. Metodologi

- Lingkungan Game

Parameter	Nilai
Grid	20 × 20 sel

Panjang awal ular	3
Panjang maksimum (win)	400 (mengisi seluruh grid)
Aksi	4 diskrit: UP, RIGHT, DOWN, LEFT
Aksi terlarang	<i>180° reversal</i> (implisit, agent tidak melihatnya sebagai opsi)
Kecepatan ular	1 sel / step

- Perbedaan Skenario

Level	Static	Dynamic	Deskripsi
1	1	0	1 Makanan Statis
2	4	0	4 Makanan Statis
3	0	1	1 Makanan Dinamis
4	0	4	4 Makanan Dinamis
5	3	2	Kombinasi 3 Makanan Statis dan 2 Makanan Dinamis

- State Space

A. Tensor (8x20x20)

Ch	Nama	Encoding
0	Wall	Batas absolut arena
1	Decaying Body	Kepala=1.0 → ekor≈0.0 (linear)
2	Static Food	1.0 di sel makanan statis
3	Dynamic Food	Posisi saat ini=1.0, posisi sebelumnya=0.5
4	Head Direction	1.0 di sel 1 langkah ke arah heading
5	Food Direction	1.0 di 4 sel sekitar kepala yang berisi makanan
6	Relative Danger	STRAIGHT/LEFT/RIGHT (relatif heading) yang berbahaya

7	Snake Length	Broadcast len/400 ke seluruh sel
---	--------------	----------------------------------

B. 12bit

Bits 0–3: obstacle wall & body diarah [ATAS,KANAN,BAWAH,KIRI]

Bits 4–7: body proximity (1 langkah ke depan)

Bits 8–11: relative food direction (dx/dy)

- **Action Space**

Diskrit: UP, RIGHT, DOWN, LEFT (3 Arah relative dari arah kedepan kepala)

Reversal 180° tidak tersedia secara fisik: tubuh ular secara langsung menghalangi

Tick skipping: makanan dinamis bergerak 2 dari setiap 3 step untuk memastikan catchable pursuit.

- **Arsitektur**

Varian	Extractor	Head
PPO-sptmp	CNN×2 → Transformer Encoder (8-head) → CLS pool	Actor + Critic MLP (64→64)
PPO-12bit	MLP (12→64→64→64)	Actor + Critic MLP (64→64)
DQN-sptmp	CNN×2 → Transformer Encoder (8-head) → CLS pool	Q-head Linear (64→4)

- **Sistem Reward**

- **Distance-based shaping:** +0.4 jika mendekat, -0.3 jika menjauh (Euclidean ke makanan terdekat)
- **Food reward:** +11 (statis), +10 (dinamis), *diskalakan* oleh REWARD_MOD sesuai panjang tubuh
- **Collision penalty:** -10, *diperberat* seiring pertumbuhan (REWARD_MOD[0] = 0.005)
- **Time penalty:** -0.002 per step (efisiensi ringan)
- **Milestone bonus:** setiap kelipatan 3 sel: $2.0 \times (1 + 0.1 \times \text{index_milestone})$
- **Stagnation penalty:** -0.3 setelah 15 langkah tanpa progress jarak
- **Encircle penalty:** -0.5 saat kepala terperangkap *flood-fill* oleh tubuh sendiri

Reward	Nilai	Metode
Approach (REWARD_APPROACH)	Jarak Euclidean kepala ke makanan <i>nearest static target</i> berkurang di step ini. Bekerja terhadap single	+0.4

	committed target (TARGET_FOOD_ENAB LED=true) untuk menghindari swap-confound; <i>tidak</i> termasuk makanan dinamis (DISTANCE_INCLUDE _DYNAMIC=false) agar tidak memberi kredit saat makanan yang mendekat bukan hasil aksi ular.	
Away (REWARD_AWAY)	Jarak Euclidean kepala ke target <i>bertambah</i> di step ini. Simetris dengan Approach sebagai penalty eksplisit untuk gerakan menjauh.	-0.3
Eat Static (REWARD_EAT_STATI C)	Trigger ketika kepala menduduki sel makanan statis. Dikalikan dengan growth factor: $\text{REWARD_EAT_STATIC} \times (1 + \min(\text{REWARD_MOD}[1] \times \text{len}, \text{REWARD_MOD_CAP}[1]))$ Default REWARD_MOD[1]=0.0 05, REWARD_MOD_CAP[1] =1.0 → reward tumbuh dari 11.0 (panjang 3) hingga 22.0 (panjang ≥ 200).	+11.0 → +22.0
Eat Dynamic (REWARD_EAT_DYNAM IC)	Trigger ketika kepala menduduki sel makanan dinamis. Growth factor lebih lambat (REWARD_MOD[2]=0. 0025) untuk menandai tingkat kesulitannya.	+10.0 → +20.0
Collision (REWARD_COLLISION)	Trigger pada tabrakan dinding atau tubuh sendiri. Dikalikan dengan penalty	-10.0 → -20.0

	amplifier 1 + $\min(\text{REWARD_MOD}[0] \times \text{len}, \text{REWARD_MOD_CAP}[0])$) agar agen lebih konservatif di late-game saat reward eating juga meningkat.	
Time (REWARD_TIME)	Diberikan setiap step regardless of action. Tekanan minimal untuk efisiensi, kecil agar tidak menghukum gerakan aman di late-game.	-0.002
Milestone Bonus (REWARD_MILESTONE)	Trigger ketika $\text{len}(\text{snake})$ melewati kelipatan $\text{REWARD_MILESTONE_INTERVAL}=3$. Formula: $\text{REWARD_MILESTONE_MOD} \times (1 + \text{REWARD_MILESTONE_GROWTH} \times \text{milestone_index})$. Default: kelipatan 5, 8, 11, 14, 17, ... masing-masing menghasilkan bonus 2.2, 2.4, 2.6, 2.8, 3.0, ... (akumulatif). Mendorong pertumbuhan melampaui satisficing attractor.	+2.2 (per milestone)
Stagnation Penalty (STAGNATION_PENALTY)	Trigger ketika kepala <i>tidak mengurangi</i> jarak ke target selama $\text{STAGNATION_THRESHOLD}=15$ langkah berturut-turut. Counter direset ketika jarak berkurang. Menangani <i>orbit attractor</i> (mengitari target).	-0.3 per step setelah threshold
Encircle Penalty (ENCIRCLE_PENALTY)	Per-step cost ketika kepala ular terperangkap di dalam <i>closed loop</i> yang dibentuk tubuhnya	-0.5 per step saat encircled

	sendiri. Dideteksi via flood-fill dari kepala: jika <i>region</i> yang terhubung tidak termasuk makanan dan area < threshold, dianggap encircled. Menangani <i>self-trap</i> failure mode.	
--	---	--

- **Hyperparameter**

Parameter	PPO	DQN
total_timesteps	500 000	200 000
learning_rate	1e-4 (linear → 0.1×)	5e-4 (linear → 0×)
n_envs cap	CPU count	4
n_steps / batch_size	2048 / 256	-
batch_size	-	64
gamma	0.99	0.99
gae_lambda	0.95	-
clip_range	0.2 → 0.05×	-
ent_coef	0.018	-
buffer_size	-	100 000
learning_starts	-	1 000 (0 saat resume)
exploration_fraction	-	0.1 (1.0 → 0.05)
target_update_interval	-	1 000

- **Penilaian**

Parameter	Jenis	Detail
-----------	-------	--------

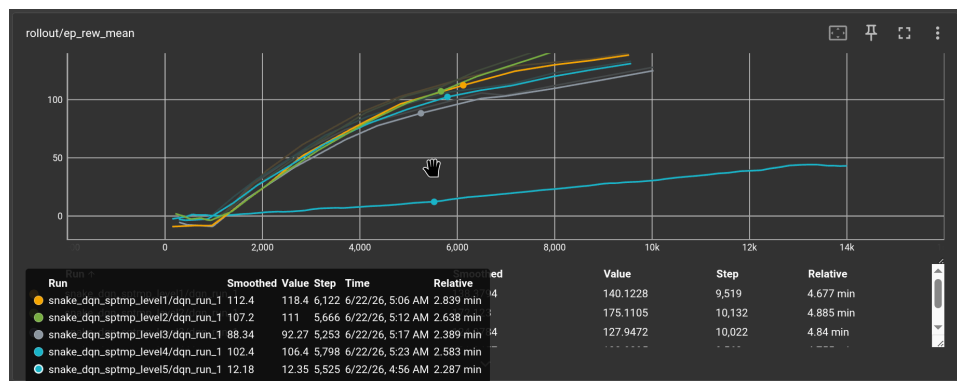
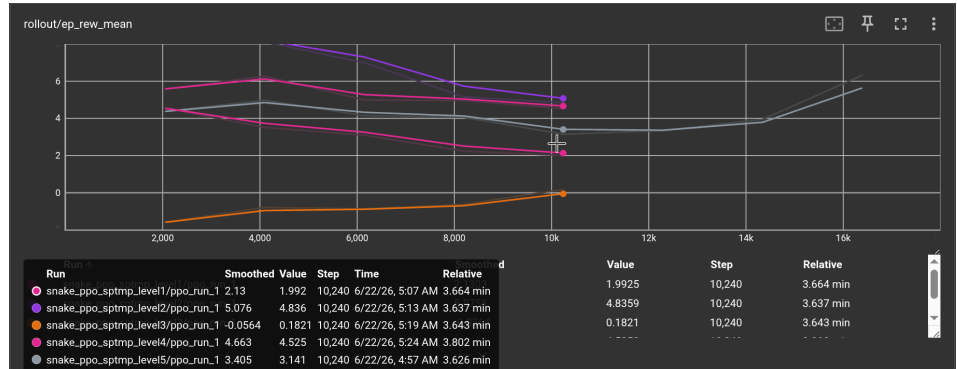
rollout/snake_length_mean	Skill	Rata-rata panjang ular saat episode berakhir. Sinyal terpenting Snake.
rollout/ep_len_mean	Survival	Rata-rata langkah sebelum episode berakhir.
rollout/eating_rate	Binary success	Fraksi episode yang makan ≥ 1 makanan.
rollout/steps_per_food	Efficiency	Rata-rata step untuk konsumsi 1 makanan. Episode 0-food di-skip. ↓ = lebih efisien.
rollout/exploration_rate	Exploration	DQN: ϵ -greedy saat ini. PPO: lihat train/entropy_loss.
rollout/snake_length_max	Peak skill	Panjang ular terbaik dalam window — untuk mendeteksi best-case skill saat rata-rata didominasi kematian dini.
train/entropy_loss (PPO)	Exploration	Penanda kebijakan masih eksploitatif vs eksploratif.
train/approx_kl (PPO)	Update magnitude	Sehat < 0.05. Spike > 0.1 → penyebab osilasi reward.
train/loss, train/q_value (DQN)	Q-network health	Watch for divergence / over-estimation.

- **Eksperimen**

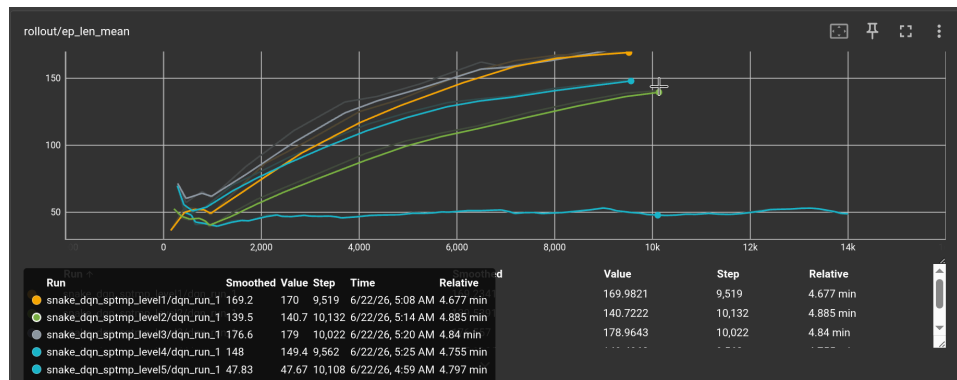
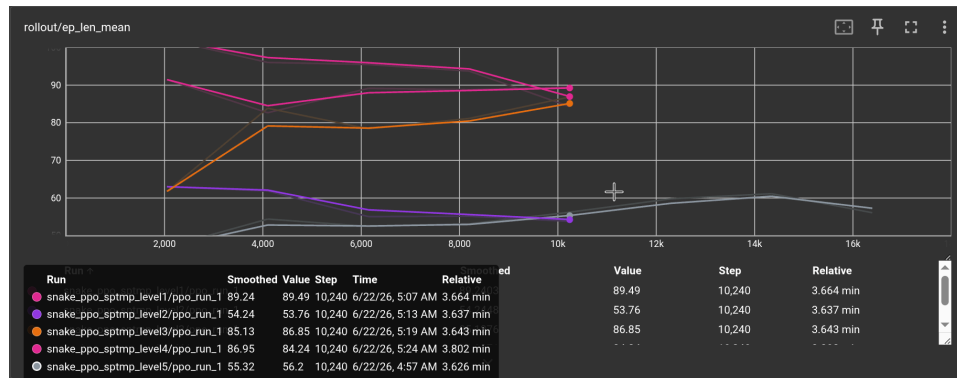
- **Curriculum study** - pelatihan sekuensial Level 1→5 (*continuous learning* via load_path).
- **Architecture ablation** - toggle use_attention=False pada Level 5 untuk isolasi peran Transformer.
- **Cross-comparison:**
 - *Algorithm effect:* PPO-sptmp vs DQN-sptmp · PPO-12bit vs DQN-12bit
 - *Architecture effect:* PPO-sptmp vs PPO-12bit · DQN-sptmp vs DQN-12bit

4. Hasil Eksperimen

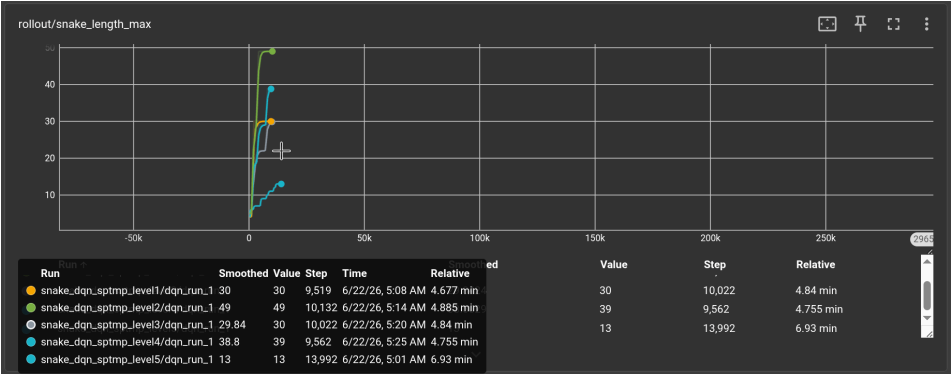
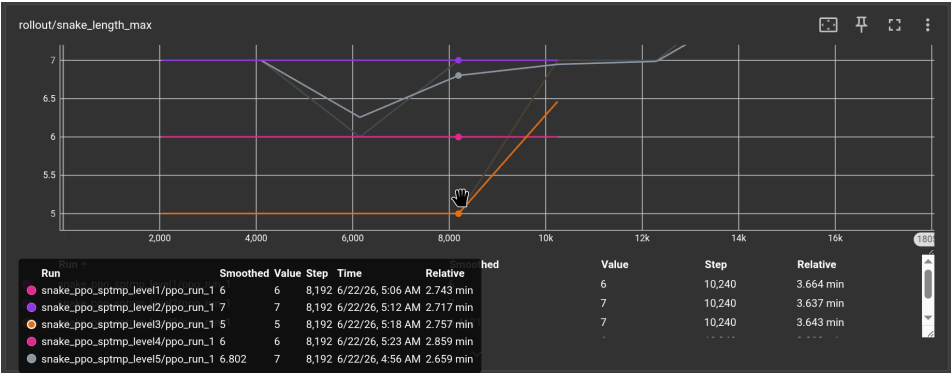
- Curriculum Study
 - Reward Curve



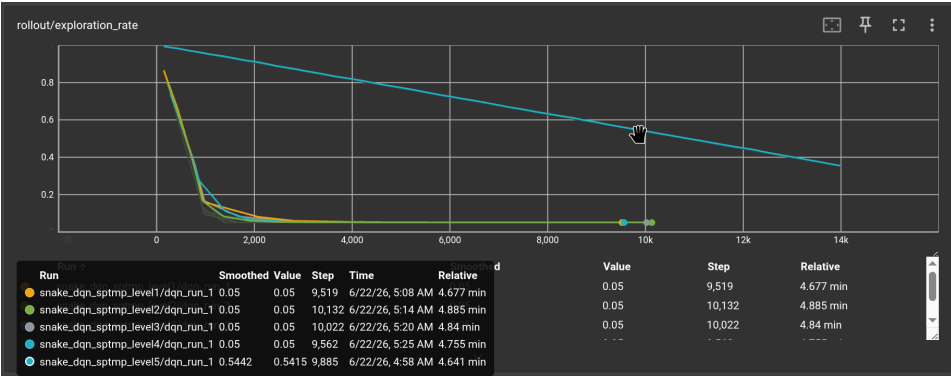
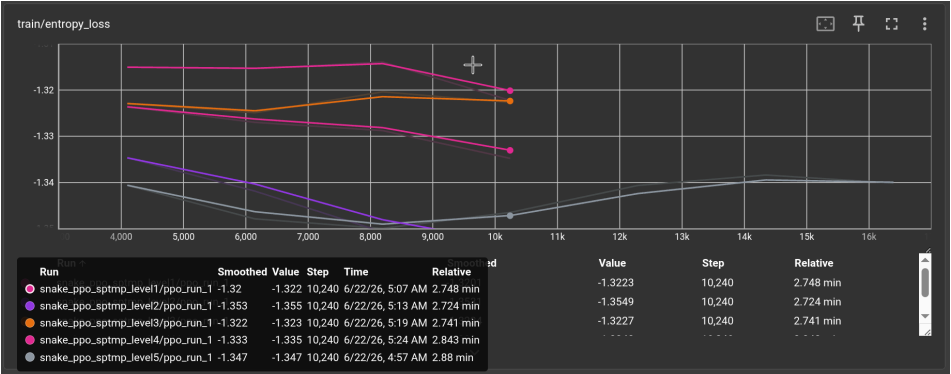
- Lama Hidup



- Panjang Ular Max

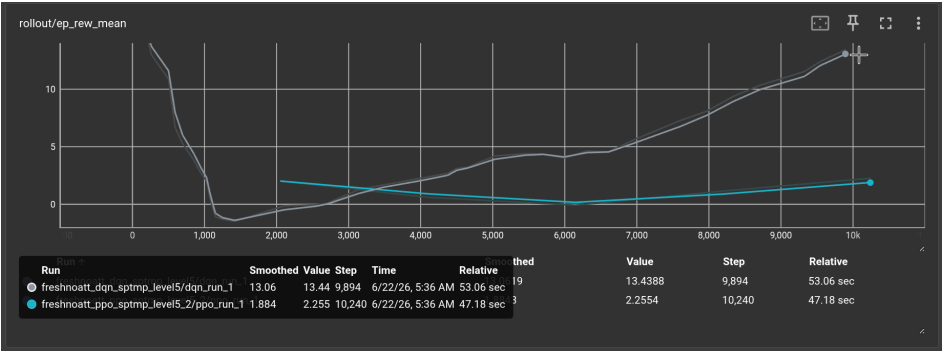


○ Entropy Loss & Exploration Loss

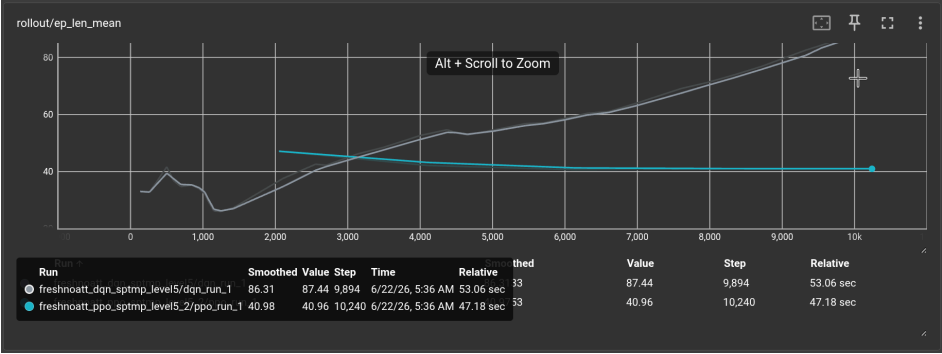


● Architecture Ablation

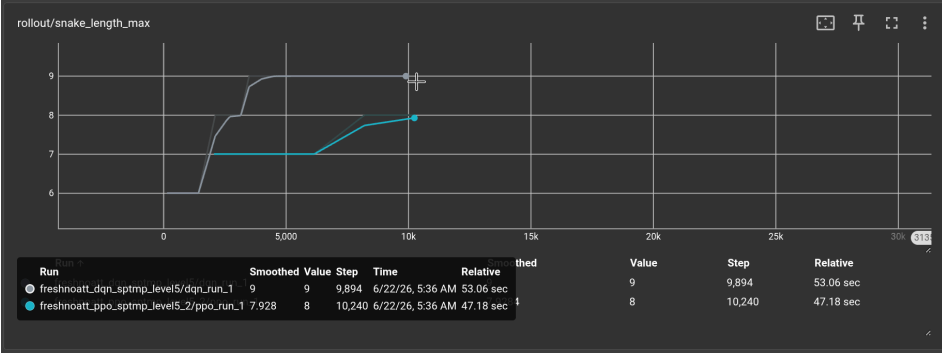
○ Reward Curve



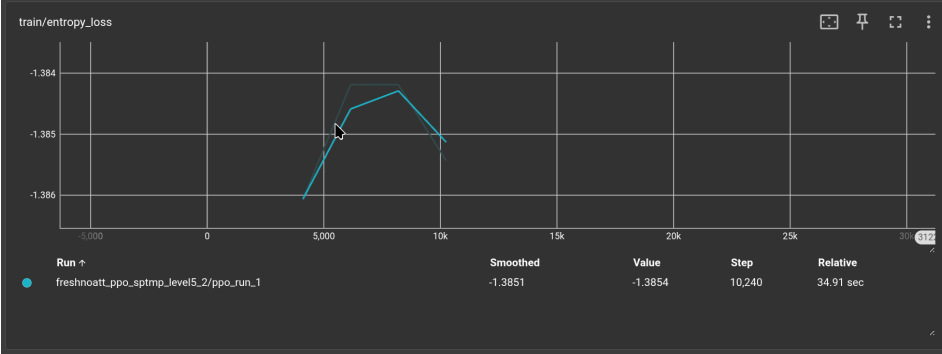
○ Lama Hidup

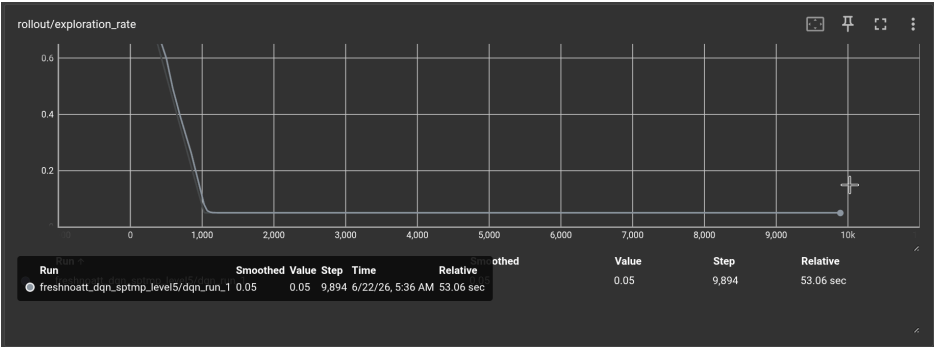


○ Panjang Ular Max

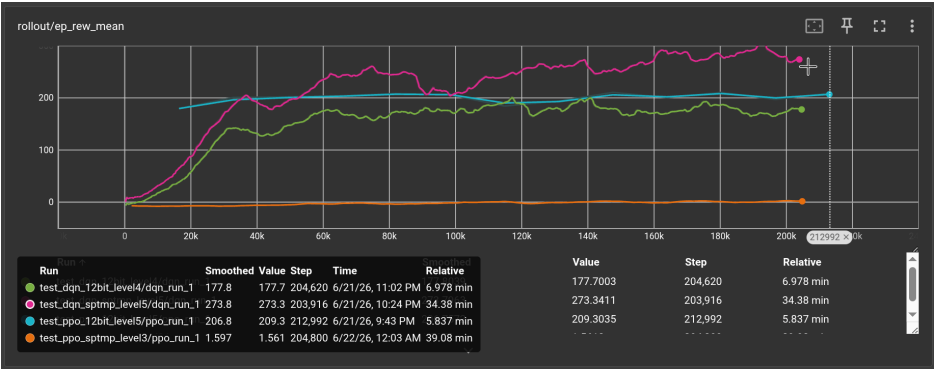


○ Entropy Loss & Exploration Rate

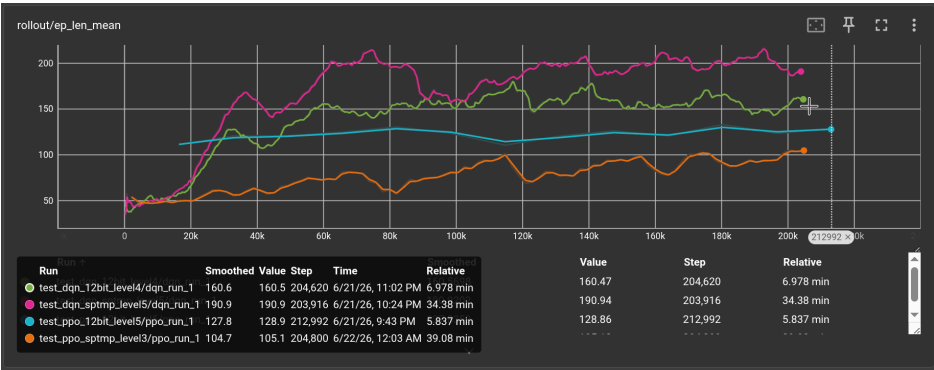




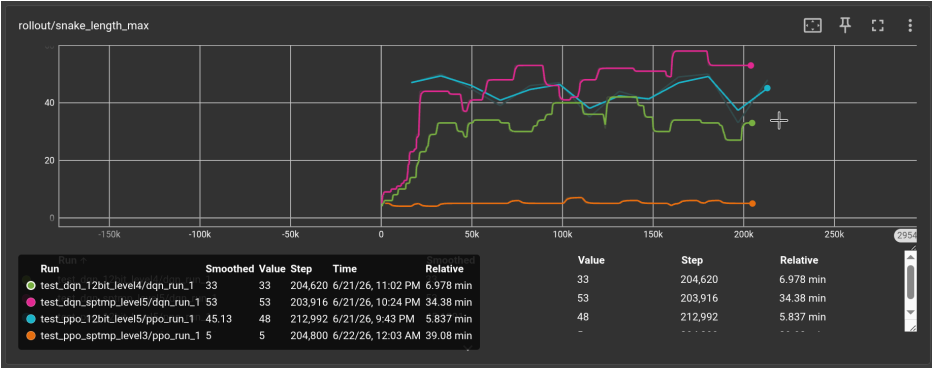
- Cross-comparison
 - Reward Curve



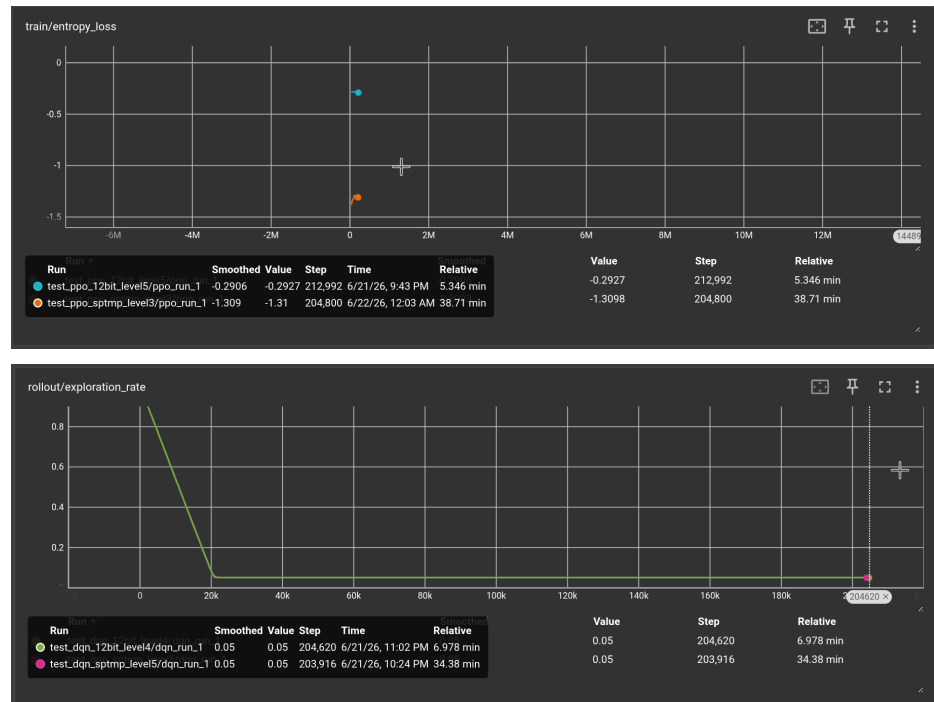
- Lama Hidup



- Panjang Ular Max



- Entropy Loss & Exploration Rate



5. Analisis dan Pembahasan

• Curriculum Study

Dilakukan 10 Percobaan pada algoritma PPO dan DQN dengan 2 jenis bentuk state space yaitu Spatiotemporal dan 12bit. Berdasarkan hasil percobaan algoritma DQN dengan kedua jenis state space dan PPO dengan 12bit berkembang dengan baik ditunjukkan oleh penurunan nilai entropy loss dan exploration rate yang menunjukkan bahwa agent telah belajar dan dapat memahami tujuan dari game, nilai entropy loss dan exploration rate juga menunjukkan pemilihan keputusan oleh agent yang kian lebih deterministik.

Terdapat permasalahan dengan PPO Spatiotemporal dimana nilai entropy loss yang masih berada jauh diatas menunjukkan bahwa agent masih berada dalam tahap eksplorasi dan pemilihan keputusan arah bersifat acak. Bukan berarti hal buruk karena dari metriks lain menunjukkan bahwa terdapat peningkatan yang menunjukkan bahwa agent sedang belajar dan mungkin membutuhkan waktu yang lebih lama.

• Architecture Ablation

Berdasarkan percobaan algoritma DQN belajar dengan cepat memahami tujuan game Exploration Rate menurun lebih cepat 1000 step dibandingkan dengan percobaan sebelumnya dengan komponen attention block(transformer encoder). Sedangkan pada PPO memiliki masalah yang sama dengan percobaan sebelumnya dengan nilai entropy loss yang sangat tinggi mungkin membutuhkan waktu pembelajaran yang lebih lama.

• Cross-comparison

Berdasarkan percobaan hasil training oleh algoritma PPO dengan 12bit menghasilkan reward curve yang lebih stabil dibandingkan DQN dengan kedua jenis state space. Menunjukkan perbedaan yang sedikit oleh kedua

implementasi jenis state space, tetapi pada DQN spatiotemporal lebih unggul dari 12bit.

6. Kesimpulan

Hasil percobaan pada 3 kategori menunjukkan keunggulan dan salah satu permasalahan pada implementasi bentuk observasi state space berupa spatiotemporal yang berupa tensor $M \times N \times 8$ (terdapat penambahan 4 channel dari proposal). Hasil percobaan menunjukkan kelebihan pada salah satu algoritma dengan hasil aksi yang lebih baik secara metrik, implementasi awal dari spatiotemporal adalah untuk memberi agent pengetahuan atas bentuk tubuhnya sendiri sehingga dapat menghindari sebuah deadlock saat agent terkunci oleh tubuhnya sendiri. Tetapi berdasarkan observasi pelatihan agent masih sering terkurung oleh dirinya sendiri, nilai pada metrik yang masih memperlihatkan kemungkinan untuk berkembang menunjukkan pelatihan pada kurang dari 100 Ribu steps tidak cukup untuk menunjukkan performa asli dari algoritma.

7. Daftar Pustaka

- Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*.
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR*.
- Towers, M., et al. (2023). Gymnasium. *Zenodo*.
- Deep Q-Snake: Deep Q-Snake: An Intelligent Agent Mastering the Snake Game with Deep Reinforcement Learning — paper baseline referensi (tersimpan di document/).
- Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2015). Prioritized Experience Replay. *ICLR*.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

8. Resource

[Codebase Repository](#)
[Presentation](#)